Київський національний університет ім. Тараса Шевченка

Кафедра мережевих та інтернет технологій

Лабораторна робота № 2

Дисципліна: Системне програмування

Тема: Паралелізм даних та паралелізм задач (ч. 1)

Виконав: Студент групи МІТ-41

Полюхович Андрій

Завдання:

1. Реалізуйте програму на C#, яка створює дві задачі (Task) для паралельного виконання:
   - перша задача виводить числа від 1 до 10 з паузою 200 мс;
   - друга задача виводить літери від A до J з паузою 200 мс.

Використайте методи Start() та WaitAll()..

Виконання:

Program.cs

```
using System;
using System.Threading;
using System.Threading.Tasks;

namespace Lab2_Task1
{
    class Program
    {
        static void PrintNumbers()
        {
            for (int i = 1; i <= 10; i++)
            {
                Console.WriteLine($"Number: {i}");
                Thread.Sleep(200);
            }
        }

        static void PrintLetters()
        {
            // Виводимо літери від A до J
            for (char c = 'A'; c <= 'J'; c++)
            {
                Console.WriteLine($"Letter: {c}");
                Thread.Sleep(200);
            }
        }

        static void Main(string[] args)
        {
            // Створення задач
```

```
            Task task1 = new Task(PrintNumbers);
            Task task2 = new Task(PrintLetters);

            // Запуск задач на виконання
            task1.Start();
            task2.Start();

            // Очікування завершення обох задач
            Task.WaitAll(task1, task2);

            Console.WriteLine("Both tasks finished execution.");
            Console.ReadLine();
        }
    }
}
```

Результат:

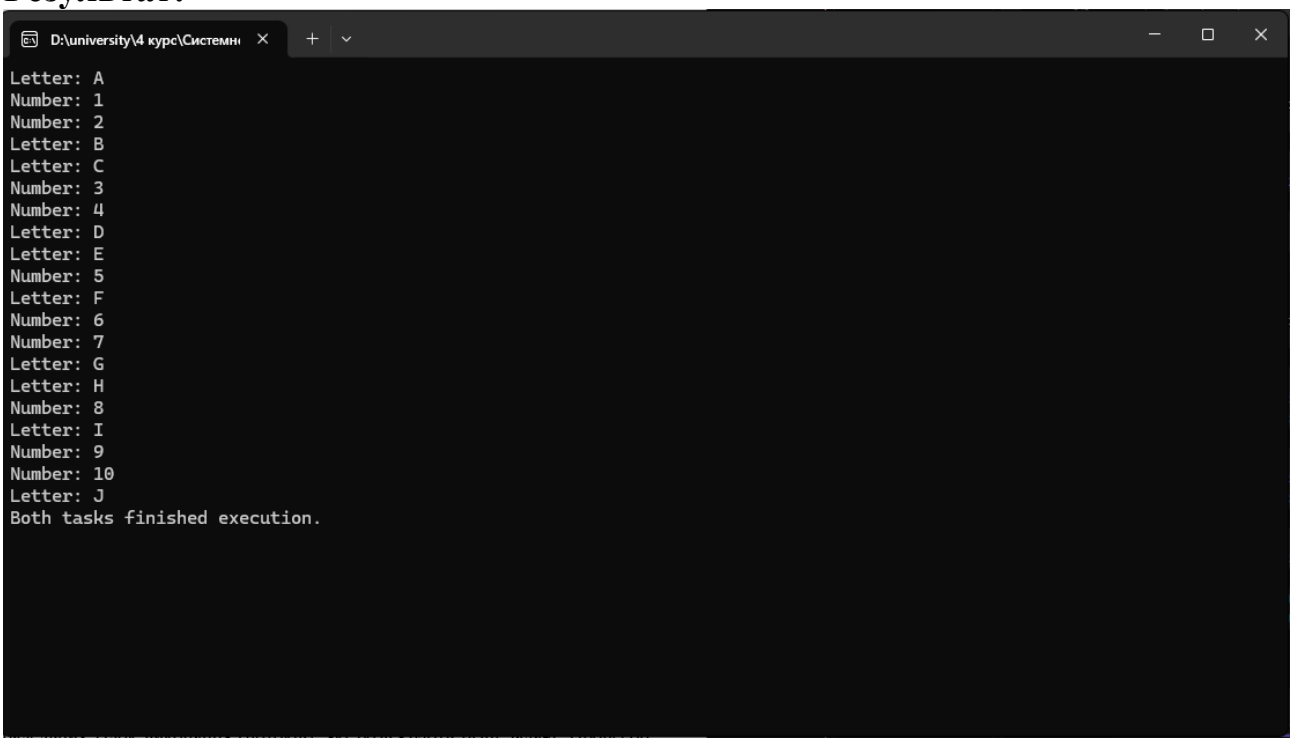

Рисунок 1.1 – Результат виконання завдання 1

Це програма демонструє найпростіший спосіб запустити два шматки коду паралельно за допомогою бібліотеки TPL (Task Parallel Library). Одна частина виводить числа, інша — літери.

Як це працює: Ми створюємо два об'єкти класу Task, передаючи їм імена методів (PrintNumbers та PrintLetters). Далі викликається метод .Start() для кожної задачі, після чого вони починають працювати одночасно (операційна система сама розподіляє їх по ядрах процесора).

Найважливіший рядок тут — Task.WaitAll(task1, task2). Він ставить головну програму "на паузу" і не дає їй закритися доти, доки обидві паралельні задачі не виконають свою роботу до кінця

Питання для звіту: поясніть різницю між класами Thread та Task.

Клас Task відрізняється від класу Thread тим, що він є абстракцією, що представляє асинхронну операцію. У бібліотеці TPL саме задачі (Task), а не потоки (Thread), є елементарною одиницею виконання коду. Використання TPL та класу Task забезпечує більш потужний та зручний спосіб управління паралельними процесами, дозволяючи створювати код з меншими зусиллями та більшою ефективністю.

Завдання:

2. Створіть програму, яка запускає три задачі, кожна з яких виконує підрахунок до 5. Виведіть у консоль ідентифікатор (Id) кожної задачі та поточний ідентифікатор (CurrentId) під час виконання.

Виконання:

Program.cs

```
using System;
using System.Threading;
using System.Threading.Tasks;

namespace Lab2_Task2
{
    class Program
    {
        static void CountToFive()
        {
            for (int i = 1; i <= 5; i++)
            {
                // Виводимо поточний ідентифікатор задачі
                Console.WriteLine($"[CurrentId: {Task.CurrentId}]
executing step: {i}");
                Thread.Sleep(150);
            }
        }

        static void Main(string[] args)
        {
            // Створюємо три задачі
            Task t1 = new Task(CountToFive);
            Task t2 = new Task(CountToFive);
            Task t3 = new Task(CountToFive);

            // Запускаємо їх
            t1.Start();
            t2.Start();
            t3.Start();

            // Виводимо ідентифікатори об'єктів задач
            Console.WriteLine($"Task 1 Id: {t1.Id}");
            Console.WriteLine($"Task 2 Id: {t2.Id}");
```

```
            Console.WriteLine($"Task 3 Id: {t3.Id}\n");

            // Чекаємо завершення всіх задач
            Task.WaitAll(t1, t2, t3);
            Console.ReadLine();
        }
    }
}
```

Результат:

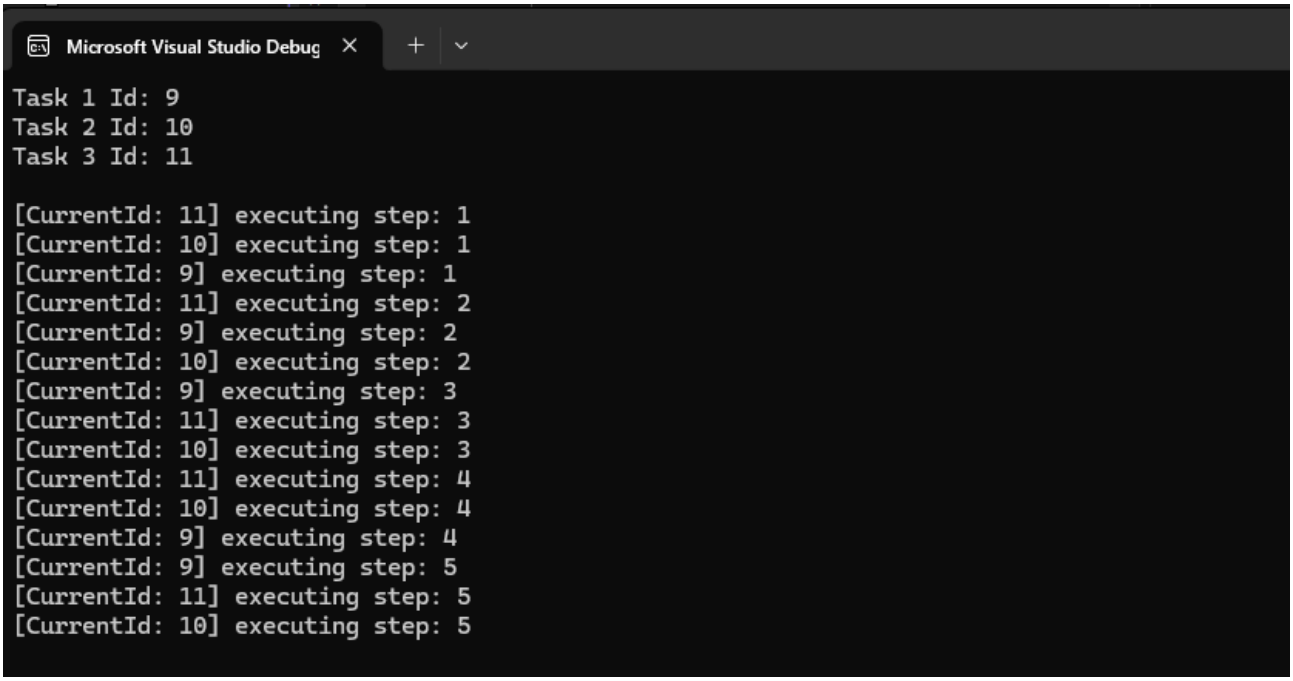

Рисунок 1.2 – Результат виконання завдання 2

Ця програма показує, що кожна задача в системі має свій унікальний "паспорт" (ідентифікатор), за яким можна слідкувати, яка саме задача зараз працює.

Як це працює: Ми створюємо три задачі, які роблять абсолютно одне й те саме — виконують метод CountToFive(). Усередині цього методу є виклик Task.CurrentId. Це спеціальна властивість, яка дозволяє задачі в момент виконання дізнатися свій власний номер у системі.

Коли запускаєм програму, бачимо, що рядки в консолі перемішуються: то працює задача №1, то задача №3, то №2. Це наочно доводить, що вони виконуються паралельно і процесор постійно між ними перемикається.

Питання для звіту: чи є ідентифікатори задач впорядкованими? Як їх можна використати для відстеження виконання?

Ні, значення ідентифікаторів задач є унікальними, але вони не є впорядкованими. Їх можна використовувати для відстеження виконання, звертаючись до властивості CurrentId, яка дозволяє виявити ідентифікатор

задачі, що виконується в даний момент часу. Це допомагає розпізнавати задачі та розуміти, яка саме паралельна гілка обробляє певну ділянку коду.

Завдання:

3. Реалізуйте програму, яка:
   - створює задачу для обчислення суми чисел від 1 до N (N вводиться користувачем).
   - використовує метод ContinueWith() для запуску другої задачі, яка виводить результат у консоль.
   - використовує лямбда-вираз для реалізації продовження.

Виконання:

Program.cs

```
using System;
using System.Threading.Tasks;

namespace Lab2_Task3
{
    class Program
    {
        static int CalculateSum(int n)
        {
            int sum = 0;
            for (int i = 1; i <= n; i++)
            {
                sum += i;
            }
            return sum;
        }

        static void Main(string[] args)
        {
            Console.Write("Enter number N: ");
            int n = int.Parse(Console.ReadLine());

            // Створення першої задачі, що повертає результат
            Task<int> sumTask = new Task<int>(() => CalculateSum(n));

            // Створення продовження за допомогою лямбда-виразу
            Task continuationTask = sumTask.ContinueWith((t) =>
            {
                Console.WriteLine($"Continuation task: The sum of numbers
from 1 to {n} is {t.Result}");
            });

            // Запускаємо першу задачу
            sumTask.Start();

            // Чекаємо завершення задачі-продовження
            continuationTask.Wait();

            Console.ReadLine();
        }
    }
```

}

Результат:

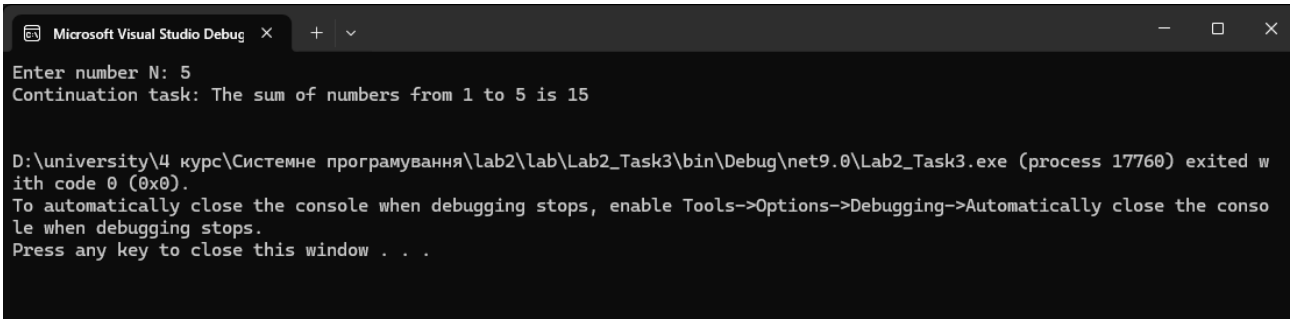

Рисунок 1.3 – Результат виконання завдання 3

Ця програма демонструє, як автоматично запустити одну задачу відразу після завершення іншої, та як передати результат обчислень з першої задачі в другу.

Як це працює: Спочатку ми створюємо задачу sumTask, яка обчислює суму. Оскільки вона має повернути число, ми використовуємо тип Task<int>.

Замість того, щоб змушувати головну програму чекати результат, ми використовуємо метод .ContinueWith(). Він створює другу задачу (continuationTask), яка сидить і чекає. Як тільки перша задача (sumTask) дораховує суму, вона завершується і автоматично дає старт другій задачі. Друга задача забирає готовий результат через змінну t.Result і красиво виводить його на екран.

Питання для звіту: які переваги має використання продовжень задач у порівнянні з класичним підходом синхронізації?

Продовження — це одна задача, яка автоматично починається після завершення другої задачі. Перевага полягає в тому, що процес, що викликає, виявиться заблокованим до тих пір, поки не буде отримано результат. Це означає, що для організації очікування результату не потрібно ніякої особливої синхронізації.

Завдання:

4. Розробіть програму, яка виконує три методи паралельно за допомогою Parallel.Invoke():
   - метод 1: обчислює факторіал числа.
   - метод 2: обчислює суму чисел від 1 до N.
   - метод 3: виводить повідомлення у консоль з паузою 300 мс..

Виконання:

Program.cs

```
using System;
using System.Threading;
```

```
using System.Threading.Tasks;

namespace Lab2_Task4
{
    class Program
    {
        // Метод тепер приймає параметр n
        static void CalculateFactorial(int n)
        {
            long factorial = 1;
            for (int i = 1; i <= n; i++) factorial *= i;
            Console.WriteLine($"Factorial of {n} = {factorial}");
        }

        // Метод тепер приймає параметр n
        static void CalculateSum(int n)
        {
            int sum = 0;
            for (int i = 1; i <= n; i++) sum += i;
            Console.WriteLine($"Sum of numbers from 1 to {n} =
{sum}");
        }

        // Метод приймає кількість повторень виводу повідомлення
        static void PrintMessage(int count)
        {
            for (int i = 0; i < count; i++)
            {
                Console.WriteLine("Message from method 3...");
                Thread.Sleep(300); // пауза 300 мс
            }
        }

        static void Main(string[] args)
        {
            // 1. Спочатку зчитуємо всі дані в головному потоці
            Console.Write("Enter a number for the factorial: ");
            int factNum = int.Parse(Console.ReadLine());

            Console.Write("Enter a number for the sum: ");
            int sumNum = int.Parse(Console.ReadLine());

            Console.Write("Enter the number of messages to print:
");
            int msgCount = int.Parse(Console.ReadLine());

            Console.WriteLine("\nStarting methods via
Parallel.Invoke()...\n");

            // 2. Паралельне виконання методів з використанням
лямбда-виразів,
            // щоб передати введені змінні як аргументи
            Parallel.Invoke(
                () => CalculateFactorial(factNum),
```

```
                () => CalculateSum(sumNum),
                () => PrintMessage(msgCount)
            );

            Console.WriteLine("\nAll methods finished execution.");
            Console.ReadLine();
        }
    }
}
```

Результат:

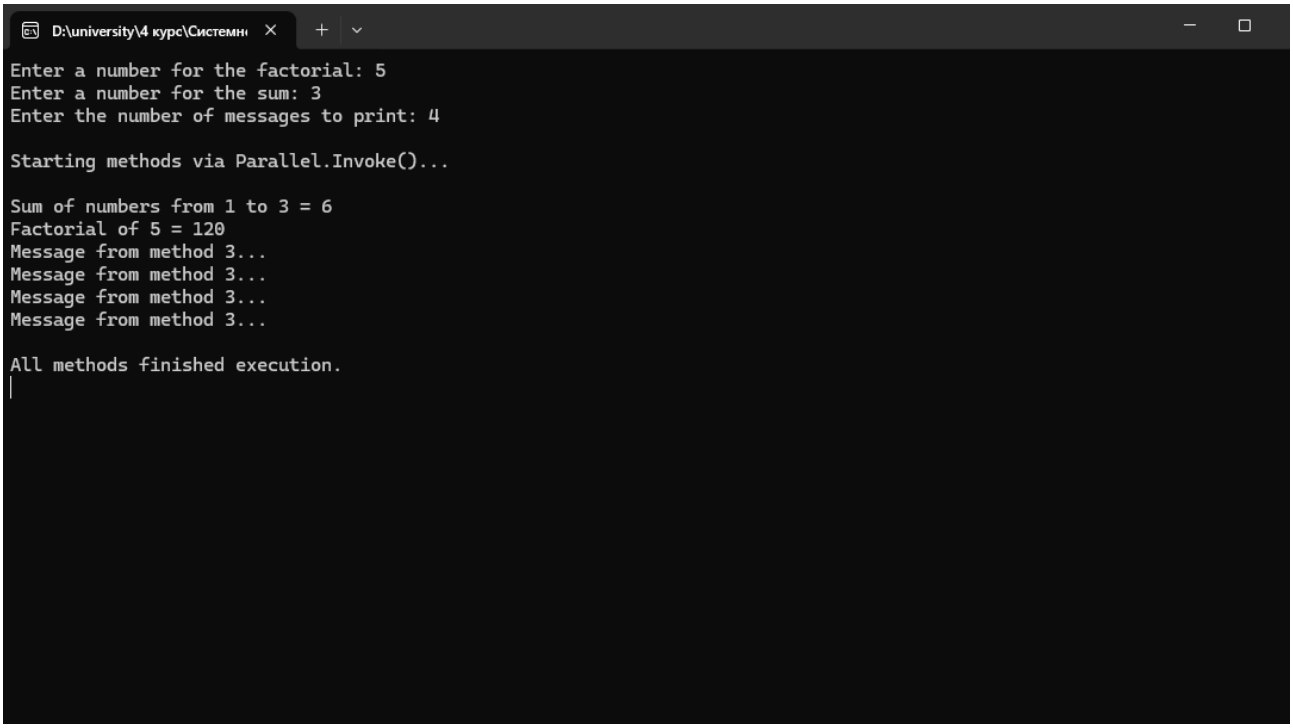

Рисунок 1.4 – Результат виконання завдання 4

Ця програма показує найшвидший спосіб запустити кілька незалежних функцій одночасно без ручного створення об'єктів Task.

Як це працює: Спочатку головний потік запитує у тебе три числа. Це робиться заздалегідь, щоб паралельні потоки потім не билися між собою за право читати з клавіатури.

Далі ми викликаємо Parallel.Invoke(...). Всередині ми передаємо три лямбда-вирази (конструкції типу () => Метод(число)), що дозволяє нам передати твої введені дані прямо всередину методів. Parallel.Invoke сам бере ці три методи, розкидає їх по різних потоках, запускає одночасно, а потім самостійно чекає, поки найдовший з них не завершить роботу. Тільки після цього програма йде далі.

Питання для звіту: які переваги має використання Parallel.Invoke() у порівнянні зі створенням окремих задач?

Метод Invoke() масштабує виконання коду, використовуючи доступні процесори, якщо є така можливість. Крім того, метод Invoke() спочатку ініціює

виконання, а потім очікує завершення всіх переданих йому методів. Це, зокрема, позбавляє від необхідності (та й не дозволяє) викликати метод Wait() для кожного потоку окремо